Docker Lab 2: Running and Managing Containers

1. Introduction

Docker has become one of the most essential tools for modern application development, deployment, and operations. It uses containerization technology to package applications and all their dependencies into standardized, isolated, and portable environments. This lab focuses on the core practical operations of running Docker containers, including port publishing, overriding default commands, persistent data storage, local file sharing between host and containers, and building multi-container applications.

By completing this lab, we can understand the lifecycle of Docker containers, master data management methods in container environments, and learn how to use Docker Compose to manage complex applications. The lab follows the official Docker documentation and covers key scenarios in real-world development and deployment.

2. Experimental Environment

- Operating System: Windows 10/11
- Virtualization: WSL 2 enabled
- Software: Docker Desktop (latest version)
- Tools: Command Prompt / PowerShell / WSL Terminal
- Network: Internet connection for pulling Docker images

3. Lab Objectives

- Understand Docker images, containers, and Docker Compose
- Publish container ports to access services externally
- Override container default commands and configurations
- Persist container data using volumes
- Share local files with containers
- Deploy and manage multi-container applications

4. Experimental Principles & Background Knowledge

4.1 What is a Docker Image

A Docker image is a lightweight, standalone, executable software package that includes code, runtime, libraries, environment variables, and configuration files required to run an application. Images are read-only and serve as the template

for creating containers.

4.2 What is a Docker Container

A container is a running instance of an image. It is an isolated process with its own filesystem, network, and process space. Containers are portable, consistent, and can run on any machine that supports Docker.

4.3 What is Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications. It uses a YAML file (docker-compose.yml) to configure all services and allows one-command startup and management of the entire application stack.

5. Lab Procedures & Screenshots

5.1 publishing-ports

Port publishing allows external access to services running inside containers. By default, containers are isolated and cannot be directly accessed from the host.

docker run -d -p 8080:80 docker/getting-started

-d: runs the container in detached (background) mode.

-p 8080:80: maps host port 8080 to container port 80.

docker/getting-started: official tutorial image with a web application.

Verification:

Run docker ps to view running containers.

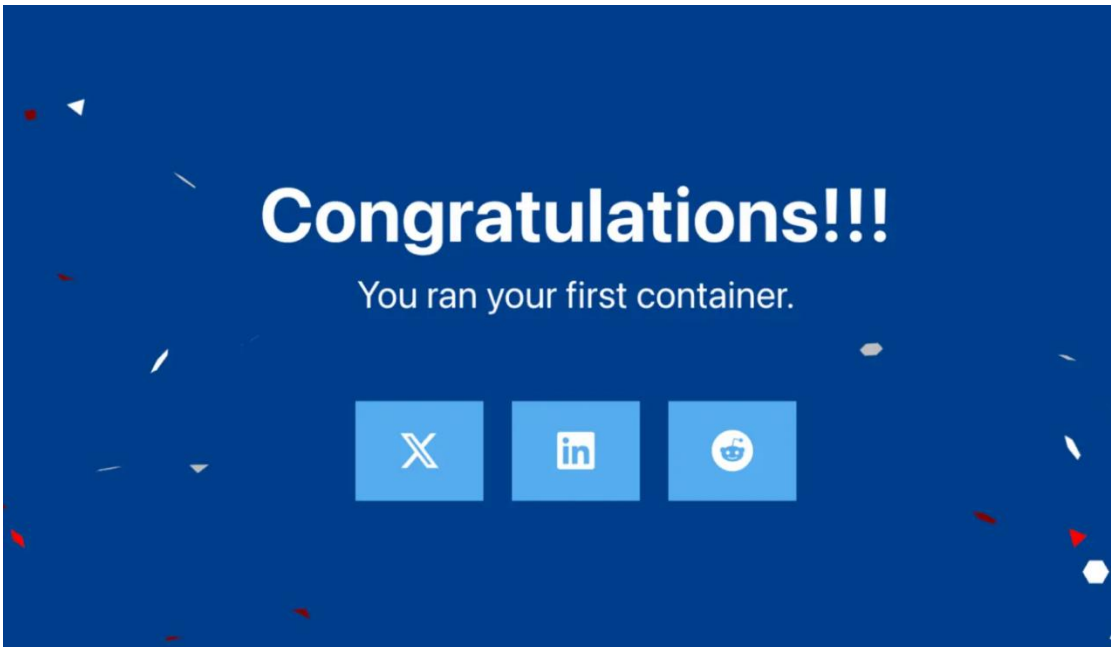
Open a browser and visit http://localhost:8080. The Docker tutorial page loads successfully, proving port publishing works.

```
PS C:\Users\36237> docker run -d -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
054715a6bffa: Pull complete
5435b2dcdf5c: Pull complete
448ea5cac5d5: Pull complete
88d1d984b765: Pull complete
4a038fd18db1: Pull complete
84e114c2bb36: Pull complete
7b5d674621c2: Pull complete
cc0cf959117b: Download complete
3eff0a97d435: Download complete
Digest: sha256:7f0adca1fc6c29c8dc49a2e90037a10ba20dc266baaed0988e9fb4d0d8b85ba0
Status: Downloaded newer image for nginx:latest
5d0da02e5a91232de5eced1b4f355c8c230c300af8b761cf55eaec5099515b64
PS C:\Users\36237>
```

```
PS C:\Users\36237> docker run -p 80 nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/10 07:23:08 [notice] 1#1: using the "epoll" event method
2026/04/10 07:23:08 [notice] 1#1: nginx/1.29.8
2026/04/10 07:23:08 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/04/10 07:23:08 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/04/10 07:23:08 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/10 07:23:08 [notice] 1#1: start worker processes
2026/04/10 07:23:08 [notice] 1#1: start worker process 29
2026/04/10 07:23:08 [notice] 1#1: start worker process 30
2026/04/10 07:23:08 [notice] 1#1: start worker process 31
2026/04/10 07:23:08 [notice] 1#1: start worker process 32
2026/04/10 07:23:08 [notice] 1#1: start worker process 33
2026/04/10 07:23:08 [notice] 1#1: start worker process 34
2026/04/10 07:23:08 [notice] 1#1: start worker process 35
2026/04/10 07:23:08 [notice] 1#1: start worker process 36
```

```
PS C:\Users\36237> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS          PORTS
 NAMES
5d0da02e5a91   nginx     "/docker-entrypoint.…"   20 minutes ago   Up 20 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
stoic_engelbart
```

☐	●	stoic_engelbart	5d0da02e5a91	nginx	8080:80	0%	23 minutes ago	



```
    目录: C:\Users\36237


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         2026/4/10     15:45                docker-compose-demo


PS C:\Users\36237> cd docker-compose-demo
PS C:\Users\36237\docker-compose-demo> @"
>> services:
>>   app:
>>     image: docker/welcome-to-docker
>>     ports:
>>       - 8080:80
>> "@ | Out-File -FilePath compose.yaml -Encoding utf8
PS C:\Users\36237\docker-compose-demo>
```

```
PS C:\Users\36237\docker-compose-demo> pwd

Path
----
C:\Users\36237\docker-compose-demo


PS C:\Users\36237\docker-compose-demo> ls


    目录: C:\Users\36237\docker-compose-demo


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         2026/4/10     15:46             84 compose.yaml
```

```
PS C:\Users\36237\docker-compose-demo> docker compose up
[+] up 12/12
 ✓ Image docker/welcome-to-docker       Pulled                                    10.0s
 ✓ Network docker-compose-demo_default  Created                                   0.0s
 ✓ Container docker-compose-demo-app-1  Created                                   0.1s
Attaching to app-1
```

5.2 overriding-container-defaults

Every Docker image has a default command or entrypoint that runs when the container starts. We can override this behavior by specifying a custom command.

--rm: automatically removes the container after it exits.

ubuntu: uses the official Ubuntu base image.

echo ...: replaces the default command with a custom message.

Observation:

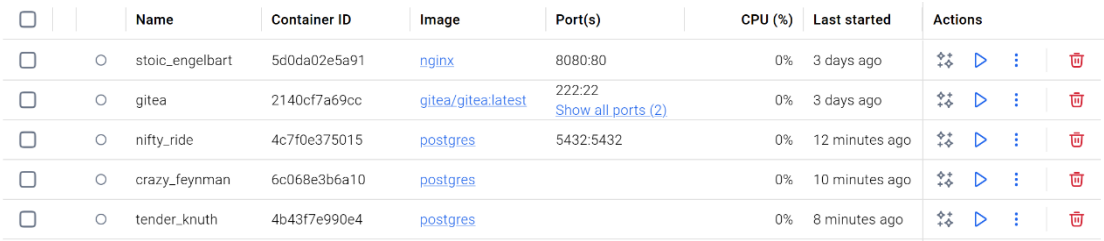
The terminal prints the custom message instead of entering the default shell, showing successful command override.

```
>> docker run -d -p 5432:5432 postgres
Unable to find image 'postgres:latest' locally
latest: Pulling from library/postgres
baf06005f9e9: Pull complete
11c182969642: Pull complete
464872ed796d: Pull complete
7a79188b0f04: Pull complete
7cec73702fc3: Pull complete
484cf2ee784f: Pull complete
2140605cc9e3: Pull complete
b21c9fd89a08: Pull complete
fecac0093525: Pull complete
8d02d282ee5a: Pull complete
f4deb76cb231: Pull complete
eb4cdb4c077a: Pull complete
ffb2fc153481: Download complete
3f82016dd5ae: Download complete
Digest: sha256:52e6ffd11fddd081ae63880b635b2a61c14008c17fc98cdc7ce5472265516dd0
Status: Downloaded newer image for postgres:latest
4c7f0e3750156511f2fb172aa5586cb3cefe41ccacbd439cb46458f048d7eba8
```

```
PS C:\Users\36237> docker run -e foo=bar postgres env
HOSTNAME=6c068e3b6a10
PWD=/
HOME=/root
LANG=en_US.utf8
GOSU_VERSION=1.19
foo=bar
PG_MAJOR=18
PG_VERSION=18.3-1.pgdg13+1
SHLVL=0
PGDATA=/var/lib/postgresql/18/docker
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/lib/postgresql/18/bin
```

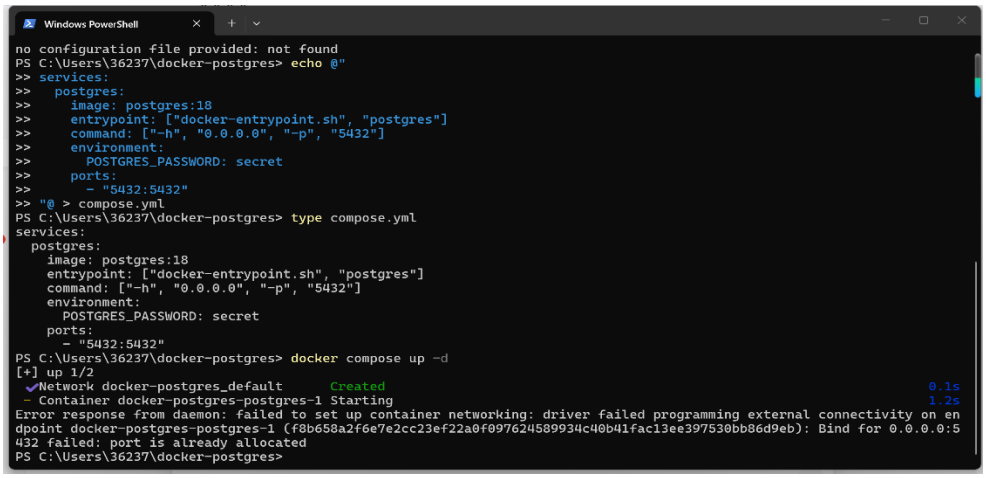
```
PS C:\Users\36237> docker run -e POSTGRES_PASSWORD=secret --memory="512m" --cpus="0.5" postgres
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.

The database cluster will be initialized with locale "en_US.utf8".
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are enabled.

fixing permissions on existing directory /var/lib/postgresql/18/docker ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default "max_connections" ... 100
selecting default "shared_buffers" ... 128MB
selecting default time zone ... Etc/UTC
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... initdb: warning: enabling "trust" authentication for local connections
initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next ti
me you run initdb.
ok
```

```
PS C:\Users\36237> docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres
b76a559c81c7bda976c95a58e788469e325779215d674fc3aebe1c64acc3099a
PS C:\Users\36237> docker run -d -e POSTGRES_PASSWORD=secret -p 5433:5432 postgres
15ccdf190a23d0aa1ddb71460c63f48cb3870a4ed4590ba43dcca331dde523e9
```

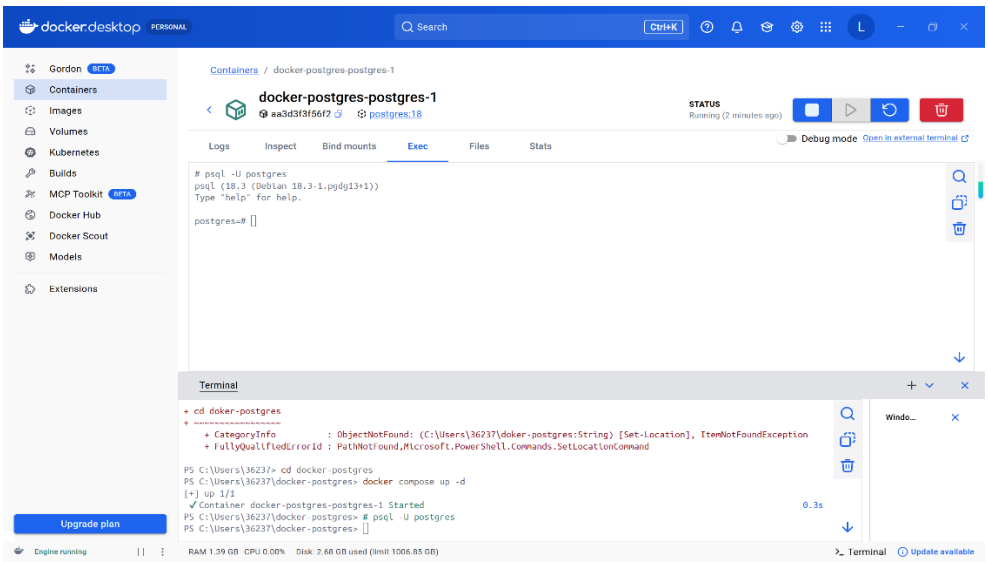


Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
stoic_engelbart	5d0da02e5a91	nginx	8080:80	0%	3 days ago	
gitea	2140cf7a69cc	gitea/gitea:latest	222:22 Show all ports (2)	0%	3 days ago	
nifty_ride	4c7f0e375015	postgres	5432:5432	0%	12 minutes ago	
crazy_feynman	6c068e3b6a10	postgres		0%	10 minutes ago	
tender_knuth	4b43f7e990e4	postgres		0%	8 minutes ago	

```
                docker network create mynetwork
b4a570d85745c1c0f611720d07b5f8e250a0111a432c55d85d8517bc4d4d7043
PS C:\Users\36237> docker network ls
NETWORK ID     NAME                      DRIVER    SCOPE
e7b87c549250   bridge                    bridge    local
dc1f8d69c4fc   docker-compose-demo_default   bridge    local
b7a1d1567c2f   host                      host      local
b4a570d85745   mynetwork                 bridge    local
28b81c335445   none                      null      local
PS C:\Users\36237> docker run -d -e POSTGRES_PASSWORD=secret -p 5434:5432 --network mynetwork postgres
0419a29b62f78f2fb3ab5042e50c3235e7c8c7ee149f38b56cd6e3fae2daea6c
```



```
Windows PowerShell
no configuration file provided: not found
PS C:\Users\36237\docker-postgres> echo @"
>> services:
>>   postgres:
>>     image: postgres:18
>>     entrypoint: ["docker-entrypoint.sh", "postgres"]
>>     command: ["-h", "0.0.0.0", "-p", "5432"]
>>     environment:
>>       POSTGRES_PASSWORD: secret
>>     ports:
>>       - "5432:5432"
>> "@ > compose.yml
PS C:\Users\36237\docker-postgres> type compose.yml
services:
  postgres:
    image: postgres:18
    entrypoint: ["docker-entrypoint.sh", "postgres"]
    command: ["-h", "0.0.0.0", "-p", "5432"]
    environment:
      POSTGRES_PASSWORD: secret
    ports:
      - "5432:5432"
PS C:\Users\36237\docker-postgres> docker compose up -d
[+] up 1/2
 ✔Network docker-postgres_default      Created                                    0.1s
 - Container docker-postgres-postgres-1 Starting                                   1.2s
Error response from daemon: failed to set up container networking: driver failed programming external connectivity on en
dpoint docker-postgres-postgres-1 (f8b658a2f6e7e2cc23ef22a0f097624589934c40b41fac13ee397530bb86d9eb): Bind for 0.0.0.0:5
432 failed: port is already allocated
PS C:\Users\36237\docker-postgres>
```



```
docker desktop PERSONAL
Gordon BETA
Containers
Images
Volumes
Kubernetes
Builds
MCP Toolkit BETA
Docker Hub
Docker Scout
Models
Extensions
Upgrade plan
Containers / docker-postgres-postgres-1
docker-postgres-postgres-1
aa3d3f3f56f2  postgres:18
STATUS
Running (2 minutes ago)
Logs  Inspect  Bind mounts  Exec  Files  Stats
Debug mode  Open in external terminal
# psql -U postgres
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.
postgres=# []
Terminal
+ cd doker-postgres
+ ~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (C:\Users\36237\doker-postgres:String) [Set-Location], ItemNotFoundException
    + FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLocationCommand
PS C:\Users\36237> cd docker-postgres
PS C:\Users\36237\docker-postgres> docker compose up -d
[+] up 1/1
 ✔Container docker-postgres-postgres-1 Started                                    0.3s
PS C:\Users\36237\docker-postgres> # psql -U postgres
PS C:\Users\36237\docker-postgres> []
Engine running
RAM 1.39 GB  CPU 0.00%  Disk: 2.68 GB used (limit 1006.85 GB)
Terminal  Update available
```

5.3 Persisting Container Data

By default, all data inside a container is deleted when the container is removed.

To preserve data permanently, we use **Docker volumes**.

Steps:

1. Create a named volume:
2. Run a container and mount the volume:
3. Verify data persistence by running a new container with the same volume:

   Result:

   The file content is still available even after the original container is deleted, proving volume persistence works.

```
postgres=# CREATE TABLE tasks(
postgres(# id SERIAL PRIMARY KEY,
postgres(# description VARCHAR(100)
postgres(# );
CREATE TABLE
postgres=# INSERT INTO tasks (description) VALUES ('Finish work'), ('Have fun');
INSERT 0 2
postgres=# SELECT * FROM tasks;
 id | description
----+-------------
  1 | Finish work
  2 | Have fun
(2 rows)
```

```
PS C:\Users\36237> docker stop db
db
PS C:\Users\36237> docker rm db
db
PS C:\Users\36237> docker run --name=new-db -d -v postgres_data:/var/lib/postgres
ql postgres:18
20e7a433997fa043457000351749f52244ba4593a72267f689fd42e878da339b
PS C:\Users\36237> docker exec -ti new-db psql -U postgres -c "SELECT * FROM task
s"
 id | description
----+-------------
  1 | Finish work
  2 | Have fun
(2 rows)
```

```
PS C:\Users\36237> docker rm -f^C
PS C:\Users\36237> docker rm -f 24ada4c0c311
24ada4c0c311
PS C:\Users\36237> docker volume rm postgres_data
postgres_data
PS C:\Users\36237> docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one contai
ner.
Are you sure you want to continue? [y/N] y
Deleted Volumes:
ea95f72f0dcaf777eec1472a9896da460107d3fb3bf394482fa25b5511dc084b
```

5.4 Sharing Local Files with Containers

During development, we often need to share local project files with containers. This is done using **bind mounts**, which map a local directory directly into a container.

"$(pwd)": current working directory on the host.

/app: target directory inside the container.

ls /app: lists files from the host inside the container.

Observation:

All local files and folders appear in the container's /app directory, enabling real-time file synchronization.

```
PS C:\Users\36237> docker run -v /HOST/PATH:/CONTAINER/PATH -it nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/15 08:11:21 [notice] 1#1: using the "epoll" event method
2026/04/15 08:11:21 [notice] 1#1: nginx/1.29.8
2026/04/15 08:11:21 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/04/15 08:11:21 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/04/15 08:11:21 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/15 08:11:21 [notice] 1#1: start worker processes
2026/04/15 08:11:21 [notice] 1#1: start worker process 29
2026/04/15 08:11:21 [notice] 1#1: start worker process 30
2026/04/15 08:11:21 [notice] 1#1: start worker process 31
2026/04/15 08:11:21 [notice] 1#1: start worker process 32
2026/04/15 08:11:21 [notice] 1#1: start worker process 33
2026/04/15 08:11:21 [notice] 1#1: start worker process 34
2026/04/15 08:11:21 [notice] 1#1: start worker process 35
2026/04/15 08:11:21 [notice] 1#1: start worker process 36
```

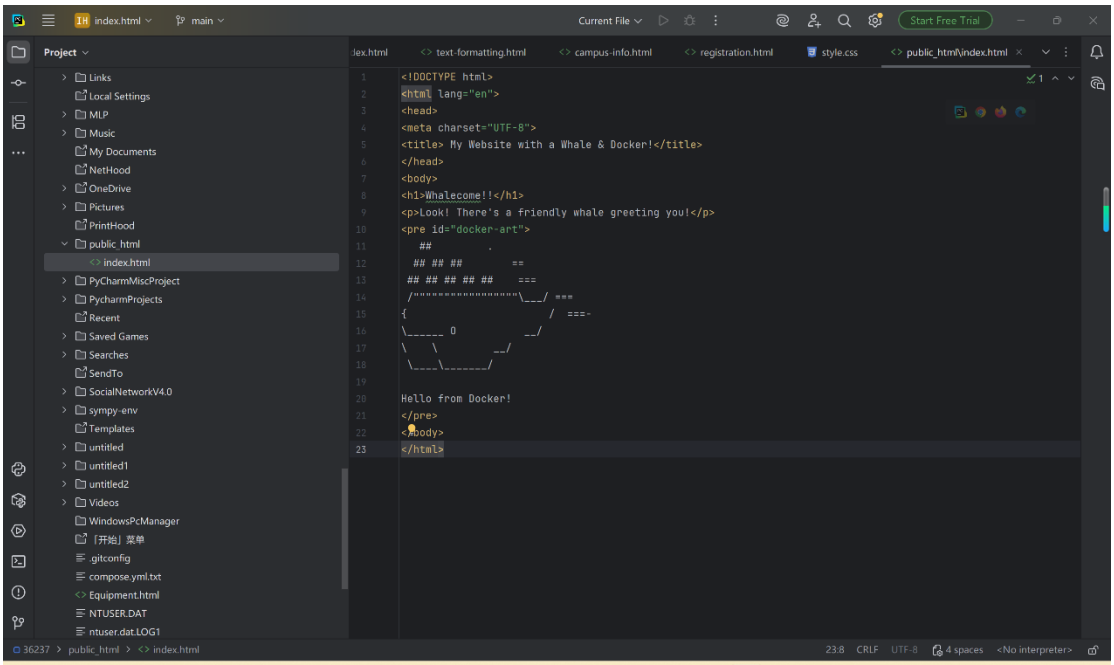
```
PS C:\Users\36237> docker run --mount type=bind,source=/HOST/PATH,target=/CONTAINER/PATH,readonly nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/15 08:11:52 [notice] 1#1: using the "epoll" event method
2026/04/15 08:11:52 [notice] 1#1: nginx/1.29.8
2026/04/15 08:11:52 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/04/15 08:11:52 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/04/15 08:11:52 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/15 08:11:52 [notice] 1#1: start worker processes
2026/04/15 08:11:52 [notice] 1#1: start worker process 30
2026/04/15 08:11:52 [notice] 1#1: start worker process 31
2026/04/15 08:11:52 [notice] 1#1: start worker process 32
2026/04/15 08:11:52 [notice] 1#1: start worker process 33
```

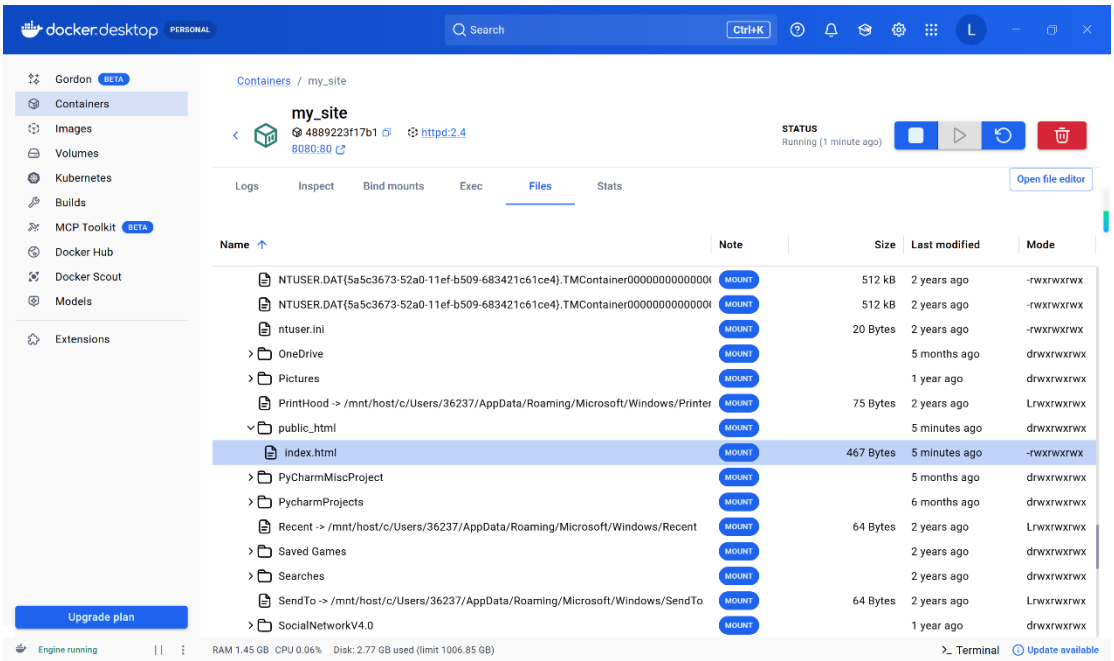
```
PS C:\Users\36237> docker run -v HOST-DIRECTORY:/CONTAINER-DIRECTORY:rw nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/15 08:13:27 [notice] 1#1: using the "epoll" event method
2026/04/15 08:13:27 [notice] 1#1: nginx/1.29.8
2026/04/15 08:13:27 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/04/15 08:13:27 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/04/15 08:13:27 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/15 08:13:27 [notice] 1#1: start worker processes
2026/04/15 08:13:27 [notice] 1#1: start worker process 29
2026/04/15 08:13:27 [notice] 1#1: start worker process 30
2026/04/15 08:13:27 [notice] 1#1: start worker process 31
2026/04/15 08:13:27 [notice] 1#1: start worker process 32
2026/04/15 08:13:27 [notice] 1#1: start worker process 33
2026/04/15 08:13:27 [notice] 1#1: start worker process 34
2026/04/15 08:13:27 [notice] 1#1: start worker process 35
2026/04/15 08:13:27 [notice] 1#1: start worker process 36
```

```
PS C:\Users\36237> docker run -d -p 8080:80 --name my_site httpd:2.4
Unable to find image 'httpd:2.4' locally
2.4: Pulling from library/httpd
8261ede62f8c: Pull complete
a838a4be55ce: Pull complete
4f4fb700ef54: Pull complete
61e1798e23ba: Pull complete
4e5144803049: Pull complete
5148763a788d: Download complete
747d2a9231eb: Download complete
Digest: sha256:bdba5c86022f2d6ad0783168b07fe80541fe45e97b434b13ba8c959a5050fb6b
Status: Downloaded newer image for httpd:2.4
c3252463d83cb85df618eb8ab62d20497c2783372595edbd301e2be01d938316
PS C:\Users\36237> curl localhost:8080
curl : 无法识别该 URI 前缀。
所在位置 行:1 字符: 1
+ curl localhost:8080
+ ~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotImplemented: (:) [Invoke-WebRequest], NotSupportedException
    + FullyQualifiedErrorId : WebCmdletIEDomNotSupportedException,Microsoft.PowerShell.Commands.InvokeWebRequestCo
   mmand
```

```
PS C:\Users\36237> curl http://localhost:8080

安全警告: 脚本执行风险
Invoke-WebRequest 可解析网页内容。解析网页时，可能会运行网页中的脚本代码。
      建议的操作:
      使用 -UseBasicParsing 开关来避免执行脚本代码。

      是否要继续?

[Y] 是(Y)  [A] 全是(A)  [N] 否(N)  [L] 全否(L)  [S] 暂停(S)  [?] 帮助 (默认值为“N”): y


StatusCode        : 200
StatusDescription : OK
Content           : <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
                    <html>
                    <head>
                    <title>It works! Apache httpd</title>
                    </head>
                    <body>
                    <p>It works!</p>
                    </body>
```

```
RawContent        : HTTP/1.1 200 OK
                    Keep-Alive: timeout=5, max=100
                    Connection: Keep-Alive
                    Accept-Ranges: bytes
                    Content-Length: 191
                    Content-Type: text/html
                    Date: Wed, 15 Apr 2026 08:18:17 GMT
                    ETag: "bf-642fce432f30...
Forms             : {}
Headers           : {[Keep-Alive, timeout=5, max=100], [Connection, Keep-Alive], [Accept-Ranges, bytes], [Content-L
                    ength, 191]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 191
```





5.5 Multi-container Applications

Most real applications consist of multiple services (e.g., web frontend, database, cache). Docker Compose manages these services together.

- Create a docker-compose.yml file defining two or more services.
- Start all services
- View running services
- Stop services

Observation:

All containers start in the correct dependency order and share a common network. They can communicate using service names as hostnames.

```
PS C:\Users\36237> git clone https://github.com/dockersamples/nginx-node-redis
Cloning into 'nginx-node-redis'...
remote: Enumerating objects: 82, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 82 (delta 22), reused 18 (delta 18), pack-reused 54 (from 1)
Receiving objects: 100% (82/82), 76.62 KiB | 1018.00 KiB/s, done.
Resolving deltas: 100% (26/26), done.
PS C:\Users\36237> cd nginx-node-redis
```

```
PS C:\Users\36237\nginx-node-redis> docker build -t nginx -f nginx/Dockerfile .
[+] Building 2.5s (8/8) FINISHED                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                  0.1s
 => => transferring dockerfile: 143B                                                                  0.0s
 => [internal] load metadata for docker.io/library/nginx:latest                                       0.1s
 => [internal] load .dockerignore                                                                     0.0s
 => => transferring context: 2B                                                                       0.0s
 => [1/3] FROM docker.io/library/nginx:latest@sha256:7f0adca1fc6c29c8dc49a2e90037a10ba20dc266baaed0988e9fb4d0d8b85ba0   0.5s
 => => resolve docker.io/library/nginx:latest@sha256:7f0adca1fc6c29c8dc49a2e90037a10ba20dc266baaed0988e9fb4d0d8b85ba0   0.1s
 => [internal] load build context                                                                     0.0s
 => => transferring context: 222B                                                                     0.0s
 => [2/3] RUN rm /etc/nginx/conf.d/default.conf                                                       0.5s
 => [3/3] COPY nginx.conf /etc/nginx/conf.d/default.conf                                              0.1s
 => exporting to image                                                                                0.9s
 => => exporting layers                                                                               0.4s
 => => exporting manifest sha256:a868d23a2368e0a3e256c411067e1efd1e496e08ec702a2c92ab5bd2a0957920     0.1s
 => => exporting config sha256:3bdaea7c3341e2f613336073c307db7ea11a773e3a1475f30f37b1f95383cdd2       0.0s
 => => exporting attestation manifest sha256:46484776095eb234e19a28c588fd077e7dd9cff881d0839a956e1d23f770947d   0.1s
 => => exporting manifest list sha256:d31d7193e54f488655ddd048433e633c8a50c4b2c267d5e0bc203ea3ea3366e2   0.1s
 => => naming to docker.io/library/nginx:latest                                                       0.0s
 => => unpacking to docker.io/library/nginx:latest                                                    0.1s
```

```
PS C:\Users\36237\nginx-node-redis> docker build -t web -f nginx/Dockerfile .
[+] Building 2.4s (8/8) FINISHED                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                0.0s
 => => transferring dockerfile: 143B                                                                0.0s
 => [internal] load metadata for docker.io/library/nginx:latest                                     0.1s
 => [internal] load .dockerignore                                                                   0.0s
 => => transferring context: 2B                                                                     0.0s
 => [1/3] FROM docker.io/library/nginx:latest@sha256:d31d7193e54f488655ddd048433e633c8a50c4b2c267d5e0bc203ea3ea3366e2   0.3s
 => => resolve docker.io/library/nginx:latest@sha256:d31d7193e54f488655ddd048433e633c8a50c4b2c267d5e0bc203ea3ea3366e2   0.1s
 => [internal] load build context                                                                   0.0s
 => => transferring context: 32B                                                                    0.0s
 => [2/3] RUN rm /etc/nginx/conf.d/default.conf                                                     0.3s
 => [3/3] COPY nginx.conf /etc/nginx/conf.d/default.conf                                            0.2s
 => exporting to image                                                                              1.3s
 => => exporting layers                                                                             0.6s
 => => exporting manifest sha256:4798edf30f183d9ac4eadb2890b359ae6d1ac0c7c8c475506095b4b8d95fe16a   0.1s
 => => exporting config sha256:8d794e3086a414eda3867a90f730374e6fbafbad53a1b3bda2fce3069b3cfbbe     0.1s
 => => exporting attestation manifest sha256:5ebbebb8ee2c78dd545e2d12aa687f39e362432573fada92861ef8712f22f61a   0.2s
 => => exporting manifest list sha256:f0e925145936abedde5edcf0ccfe139e2c356053b303678bffd381845dc81a5d   0.1s
 => => naming to docker.io/library/web:latest                                                       0.0s
 => => unpacking to docker.io/library/web:latest                                                    0.1s
PS C:\Users\36237\nginx-node-redis>
```

```
latest: Pulling from library/redis
edc4b8e535e8: Pull complete
387e0421c8da: Pull complete
e22d95bb4ed9: Pull complete
dbf833dfdfed: Pull complete
0d8ecf679ede: Pull complete
4f4fb700ef54: Pull complete
5eefc97b6afb: Download complete
048ce8724475: Download complete
Digest: sha256:1f073813b641755b70b0200da64131bbeeb4ec5b633ca67772229b49820caafa
Status: Downloaded newer image for redis:latest
f06db8ae7227679a47a6bd71d168ef8dcb4d8511e13f96b8c3058866ac3b9d06
PS C:\Users\36237\nginx-node-redis> docker run -d --name web1 -h web1 --network sample-app --network-alias web1 web
70f767a53d87c44d9806053d6ca1e9ced28cf1c9f8746532ed1a2ad138f7280e
PS C:\Users\36237\nginx-node-redis> docker run -d --name web2 -h web2 --network sample-app --network-alias web2 web
5c53f22d62bb1ce027d47c5504aeec86be6700502d7177802eb75f1a5f7c737e
PS C:\Users\36237\nginx-node-redis> docker run -d --name nginx --network sample-app  -p 80:80 nginx
84f1e6a7ea4fe12957bf527ad69080d6e2dff7ec11f121caa09285babd9f7648
PS C:\Users\36237\nginx-node-redis> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS      NAMES
5c53f22d62bb   web       "/docker-entrypoint.…"   37 seconds ago   Up 37 seconds   80/tcp     web2
f06db8ae7227   redis     "docker-entrypoint.s…"   5 minutes ago    Up 5 minutes    6379/tcp   redis
```

redis	f06db8ae7227	redis		0.27%	6 minutes ago		
web1	70f767a53d87	web		0%	1 minute ago		
web2	5c53f22d62bb	web		0%	1 minute ago		
nginx	84f1e6a7ea4f	nginx	80:80	0%	50 seconds ago		

```
[+] up 8/8
 ✔Image nginx-node-redis-nginx        Built                                   7.9s
 ✔Image nginx-node-redis-web1         Built                                   7.9s
 ✔Image nginx-node-redis-web2         Built                                   7.9s
 ✔Network nginx-node-redis_default    Created                                 0.1s
 ✔Container nginx-node-redis-web2-1   Started                                 1.2s
 ✔Container nginx-node-redis-redis-1  Started                                 1.3s
 ✔Container nginx-node-redis-web1-1   Started                                 1.1s
 ✔Container nginx-node-redis-nginx-1  Started                                 1.4s
```

6. Results and Analysis

• **Port Publishing**: Successfully exposed container services to the host. Port mapping is essential for accessing web apps, databases, and APIs.

• **Command Override**: Flexible container startup behavior allows debugging, testing, and one-time execution.

• **Data Persistence**: Volumes separate data from containers, ensuring safe data storage across container restarts and deletions.

- **File Sharing**: Bind mounts support development workflows by syncing local code into containers.
- **Multi-container Management**: Docker Compose simplifies orchestration, networking, and dependency management for microservices.

7. Conclusion

This experiment comprehensively covers the core operations of running Docker containers. We successfully practiced port publishing, command overriding, data persistence, file sharing, and multi-container deployment.

These skills are widely used in cloud computing, microservices, DevOps, and continuous integration/continuous deployment (CI/CD). By completing this lab, we have laid a solid foundation for using Docker in real development and production environments.

All commands, screenshots, and notes have been organized and uploaded to the GitHub repository as required.